user is application code (or, more precisely, the developer writing the application code) rather than a physical person, you're effectively writing an API. In fact, any code that you write can be considered an API for someone else, even if that someone is yourself.

Applying BDD principles to the actual implementation of these layers and components can help you write cleaner, better-designed, and more maintainable code. When you implement new code, you think about what data and services you need and how you'd ideally like to obtain them. If they don't exist yet, you implement your code *as if they do exist*, and then use this code as the starting point for their implementation (see figure 9.9). This is a form of the Feature Injection idea you saw in chapter 3.

For example, imagine you're working on implementing a new feature for the Flying High Human Resources application. The requirement is to export an Excel spreadsheet containing a list of employees who have a birthday in the current week. The corresponding scenario might look like this:

```
Scenario: Export staff birthdays as an Excel spreadsheet
  Given the following staff members:
  | Name  | Birthday       |
  | Joe   | 10-Mar-1980    |
  | Jill  | 18-Dec-1965    |
  | Jack  | 20-Dec-1965    |
  | Joan  | 20-Nov-1991    |
  And today is 16-Dec-2013
  When I export this week's birthday list
  Then I should obtain a spreadsheet containing the following:
  | Name  | Birthday       |
  | Jill  | 18-Dec-1965    |
  | Jack  | 20-Dec-1965    |
```

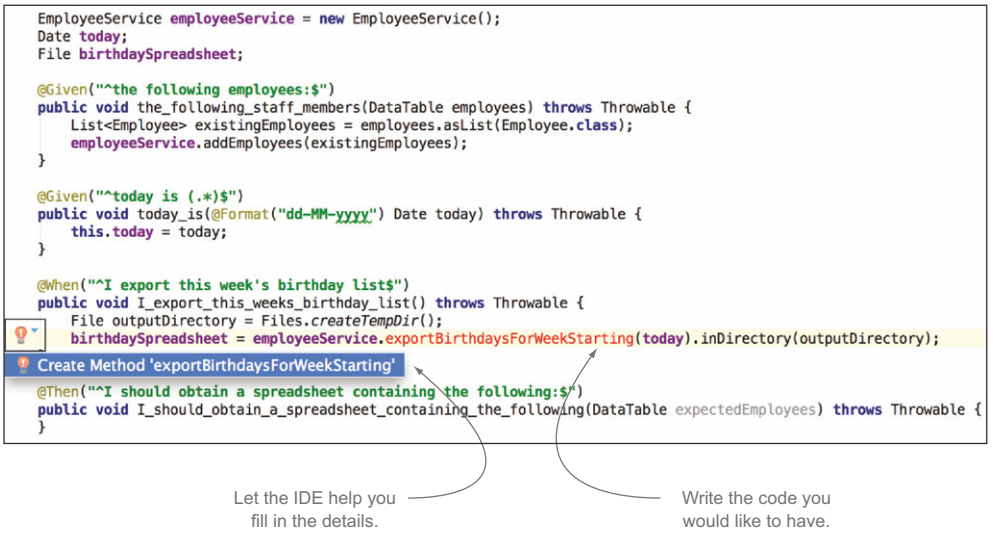


Figure 9.9 Writing the code you'd like to have. Here, you use the "quick-fix" feature available in most modern IDEs to get the IDE to generate the missing method for you.